Multi-Declaration Z Constructs — Demo

txt2tex

July 11, 2026

Background

The constructs in this file all share a single grammatical shape: a Z paragraph that introduces two or more typed variables in one declaration list, with a conjunction predicate over their projections, optionally followed by a characteristic expression. Until the multi-decl parser fix, none of them parsed natively — users wrote them as raw LaTeX. They now compile to fuzz-clean output.

$$[PlayerName, TournamentId, MatchId, VenueName]$$

Tournament

$$\underline{tournamentId} : TournamentId$$
$$venue : VenueName$$
$$tier : \mathbb{N}$$

$$tier \leq 5$$

Player

$$\underline{name} : PlayerName$$
$$ranking : \mathbb{N}$$

Match

$$\underline{matchId} : MatchId$$
$$\underline{tournament} : TournamentId$$
$$winner : PlayerName$$
$$loser : PlayerName$$

$$Ali : PlayerName$$
$$CentreCourt : VenueName$$

1. WYSIWYG line breaks for algebra operators

Long relational-algebra chains can be broken at any operator. Natural newlines work; explicit \ continuation also works. Display position wraps in array; the chain reads top-to-bottom instead of running off the page.

$$\begin{array}{l}\mathrm{Project}_{winner, venue, tier}(\mathrm{Join}(Tournament, \\ \quad (\mathrm{Project}_{tournament}(\mathrm{Restrict}_{winner = Ali}(Match)))[tournamentId / tournament]))\end{array}$$

2. Multi-decl comprehension with conjunction predicate

This is the canonical multi-typed relational-calculus form for any query that crosses two or more relations. Before the fix it required a raw LATEX block; now it parses natively.

$$\{ m : Match; t : Tournament; p : Player \mid m.tournament = t.tournamentId \land m.winner = p.name \land t.venue = CentreCourt \bullet \langle\!| winner == m.winner, venue == t.venue, tier == t.tier |\!\rangle \}$$

3. Multi-decl quantifier with conjunction predicate

'forall s : T; c : U — predicate . body' works the same way — nested Quantifier nodes covering the chained declarations.

$$\forall m : Match; t : Tournament \mid m.tournament = t.tournamentId \land t.tier \leq 2 \bullet m.winner \neq m.loser$$

4. Multi-decl quantifier without characteristic expression

When the predicate alone suffices — no '.' separator, no body expression — the comprehension defaults the characteristic tuple to the signature (Z RM §3.9).

$$\{ m : Match; t : Tournament \mid m.tournament = t.tournamentId \land t.tier \leq 2 \}$$

5. Multi-decl mu (definite description)

'mu' parses with multi-decl signatures the same way. The current generator emits nested '(\mu ... — (\mu ...))' — semantically equivalent but not Spivey-canonical. The same special-case treatment lambda just received is a follow-up for mu. Single-decl mu (shown here) emits cleanly.

$$topFinal == (\mu t : Tournament \mid t.tier = 1 \bullet t.venue)$$

6. Multi-decl lambda — Spivey-canonical form

Lambdas of more than one variable now emit as a single \lambda token with semicolon-joined Schema-Text, matching Z RM §3.12 and the Spivey convention. Previously the generator emitted nested lambdas (one per declaration).

$$roster == (\lambda m : Match; t : Tournament \mid m.tournament = t.tournamentId \bullet m.winner)$$

7. Paren-wrap fix

Fuzz requires '(\lambda ... @ ...)' — parentheses around every lambda expression, at any position. The generator previously only wrapped when the lambda appeared inside another expression ('parent is not None'). This meant abbreviation RHS, set literals, sequence displays, and top-level zed predicates emitted bare '\lambda', which fuzz rejected with "Opening parenthesis expected at symbol \lambda".

The fix: drop the 'parent is not None' guard. Wrapping is now unconditional when 'use_fuzz=True', matching how '\mu' has always been handled. All thirteen lambda-paren tests now pass without xfail markers. Section 6 above also now fuzz-typechecks cleanly.

Additional positions confirmed working: lambda as abbreviation RHS (‘f == lambda ...‘), inside a set literal (‘{lambda ...}‘), inside a sequence display (‘¡lambda ...¿‘), as a function argument (‘f(lambda ...)‘), as a tuple component (‘(lambda ..., 0)‘), and as a unary-operator argument (‘dom (lambda ...)‘).

$$f == (\lambda\, x : \mathbb{N} \bullet x + 1)$$